

Quiz 03 – Dynamic Arrays and Amortized Analysis

Estructuras de Datos, Estudiantes: Jeicob Stiven Murillo Ramos (1000334659), Andrés Santiago Morales Robayo (1011080401)

1. Dynamic Array from Scratch

b) Invariantes: 1. $0 \leq \text{size} \leq \text{capacity}$, $\text{capacity} \geq 1$, si $\text{size} = \text{capacity}$ debe duplicarse capacity , 2. $\text{data.length} = \text{capacity}$, data.length = cuántas casillas tiene físicamente el arreglo (lo sabe Java)., capacity es la variable donde se anota el mismo número, 3. capacity nunca disminuye

c) Worst-Case:

Para get y $\text{set} = \theta(1)$, solamente es validación + acceso directo. Para $\text{removeLast} = \theta(1)$, porque no se contrae, Para $\text{append} = \theta(n)$ cuando $\text{size} = \text{capacity}$ (porque crea arreglo y copia n elementos uno a uno), $\theta(1)$ en otro caso.

d) No se contradicen porque los resize caros son raros: después de duplicar a $2C$, tienes que hacer C appends baratos antes del siguiente resize . Esos baratos "pagan por adelantado" el caro.

2. Snake: una implementación aparentemente eficiente

a) REMOVE_TAIL() ejecuta $\text{size} - 1$ desplazamientos, por lo que la complejidad es de $\theta(k)$, ya que tiene que correr todos los datos un espacio atrás

b) Para cada movimiento m con longitud constante porque no come, a lo sumo hay un resize , por lo que la complejidad sería de $\theta(k) + \text{REMOVE_TAIL} : \theta(k) = \theta(mk)$

c) Falso: $\text{MOVE} = \text{append} + \text{REMOVE_TAIL}$. El amortizado de una suma es la suma de amortizados: $\theta(1) + \theta(k) = \theta(k)$. REMOVE_TAIL no es caro "a veces", es caro siempre, y no hay operaciones baratas que lo compensen

d) El paso de borrar la cola, porque sin importar cuanta longitud tenga debe correr todos los datos una casilla atrás, en cada movimiento, append cuesta solamente 1, mientras que remover la cola cuesta k

3. Redesign Challenge

a) Variables: *data*: El arreglo nativo `Position[]`, *capacity*: Largo del arreglo, *size*: Cuántos segmentos tiene la serpiente, *front*: Posición física donde está la cola.

b) Se usa la formula $\text{phys}(i) = (\text{front} + i) \bmod \text{capacity}$, donde mediante el mod se determina que no se puede pasar del tamaño de la lista

c y d) `public void addHead(Position p)`

```
public void addHead(Position p) {
    if (size == capacity) {
        resize(capacity * 2);
    }
    data[physical(size)] = p;
    size++;
}
```

public Position removeTail()

```
public Position removeTail() {
    if (size == 0) {
        throw new IllegalStateException("La serpiente esta vacia");
    }
    Position tail = data[front];
    data[front] = null;
    front = (front + 1) % capacity;
    size--;
    return tail;
}
```

public Position get(int logicalIndex)

```
public Position get(int logicalIndex) {
    if (logicalIndex < 0 || logicalIndex >= size) {
        throw new IndexOutOfBoundsException("Indice fuera de rango: " + logicalIndex);
    }
    return data[physical(logicalIndex)];
}
```

resize()

```
private void resize(int newCapacity) {
    Position[] newData = new Position[newCapacity];
    for (int i = 0; i < size; i++) {
        newData[i] = data[physical(i)];
    }
    data = newData;
    capacity = newCapacity;
    front = 0;
}
```

e) Invariantes 1. $0 \leq \text{front} < \text{capacity}$, $0 \leq \text{size} \leq \text{capacity}$, $\text{data.length} == \text{capacity}$, 2. El segmento lógico i está en $\text{data}[\text{phys}(i)]$, 3. Las casillas fuera de la serpiente son null, 4. capacity nunca baja

f)

Paso	Arreglo Físico	Lógico (tail → head)	Front	headIdx	size	capacity	resize
Inicial	[A,B]	AB	0	1	2	2	NO
N1	[_,B,C,_]	BC	1	2	2	4	SI
N2	[_,B,C,D]	BCD	1	3	3	4	NO
N3	[E,_,C,D]	CDE	2	0	3	4	NO
N4	[E,F,_,D]	DEF	3	1	3	4	NO
N5	[E,F,G,D]	DEFG	3	2	4	4	NO
N6	[_,E,F,G,H,_,_,_]	EFGH	1	4	4	8	SI

4. Amortized Analysis: demostrar, no afirmar

a) Al iniciar con tamaño 2, y solo crecer al comer manzanas (g), el tamaño llega a $2 + g$, pero en un movimiento sin comer el código agrega una cabeza con *addHead* que está antes de *removeTail*, y por ese instante tiene uno más, quedando $n_{max} < g + 3$

Cada *resize* copia un arreglo lleno, los tamaños van 2, 4, 8..., y solo se desbordan los que la serpiente alcanza a superar. Como cada tamaño es mayor que todos los anteriores juntos, la suma es menor que el doble del tamaño máximo, que es $g + 3$, entonces $copias < 2 * (g + 3) - 2 = 2g + 6 - 2 = 2g + 4$, por lo que las copias son a lo sumo el doble de las manzanas más una constante, $O(g)$.

b) **Por qué la suma es lineal:** Dado que la capacidad solo se duplica durante un *resize* (sin encogerse), cada tamaño de capacidad ocurre una única vez y obliga a copiar exactamente esa misma cantidad de elementos, pues en ese instante el tamaño iguala a la capacidad. Como cada nueva capacidad es el doble de la anterior, la suma de todas las copias previas es estrictamente menor que la última capacidad alcanzada, la cual, a su vez, siempre será menor al número máximo de elementos n_{max} .

c) $amortizado = (costo\ total\ de\ todos\ los\ addHead) / (cuántos\ addHead\ hubo)$, entonces $amortizado \leq (m + 2g + 4) / m$, en el peor caso $g = m$ porque no puede comer más que moverse, entonces $(m + 2m + 4) / m = (3m + 4) / m$, el resultado es muy pequeño, se va acercando a 3 en cantidades de movimientos muy grandes, por lo que $O(1)$.

d) m solamente marca cuantos movimientos hace, mientras que g marca el tamaño máximo y por lo tanto todas las copias, por ejemplo, un millón de movimientos sin comer ($g = 0$) producen un solo *resize* (el de N_1). Moverse mucho no llena el arreglo; crecer sí.

e) **Método del Banquero:** Se toma el precio de *addHead* como 3 y el de *removeTail* como 1, por lo que cada *addHead* normal: cuesta 1, cobra 3 y sobran 2 que van a la cuenta, para cada *addHead* con *resize*: la escritura (1) se paga con el cobro, sobran 2 que se guardan, y las copias se pagan con la cuenta que se lleva, y el *removeTail*: cuesta 1, cobro 1 y la cuenta no cambia. Debe valer 3 el *addHead* para poder cubrir el pago del *resize*, por ejemplo, luego de hacer el *resize* de 4 a 8, para llegar del valor 5 al 8 deben pasar por 3 *addHead* mas el otro *addHead* para hacer el siguiente *resize* son un total de 4 *addHead*, lo que significa que esos 4 deben dejar suficientes créditos para pagar el siguiente *resize*, por lo que cada uno debe dejar $2 + 1$ que cuesta su propia escritura.

5. The Adversarial Snake

a) Un *resize* ocurre si y sólo si **size = capacity**. La capacidad en ese momento es algún valor C_i de la sucesión ($C_i = f(C_{i-1})$; $C_0 = C_{inicial}$); el adversario tiene la capacidad de decidir cuando aparecen las manzanas pero no puede alterar el valor de **capacity** cuando se llena, siendo siempre el término siguiente de la sucesión, por lo tanto, el costo total de copiado es una cantidad que solo va a depender de n y f (del número de inserciones necesarias para llegar a n) y no de la secuencia específica de operaciones. El caso contrario (con contradicciones) sería lo que le faltaría al adversario para aumentar el número de copias y el poder forzar un nuevo crecimiento, logrando oscilar **size**.

b) Sea C_0 la capacidad inicial (ejemplo 2) para la política A la capacidad crece de uno en uno ($C_0, C_0 + 1, C_0 + 2, \dots, n$; siendo el número de resizes $= n - C_0 = \theta(n)$), para la política B tiene un crecimiento geométrico de razón 2 ($C_0, 2C_0, 4C_0, \dots, 2^k C_0 \geq n$; $k = \log_2(\frac{n}{C_0}) = \theta(\log n)$) y para la política C tiene un crecimiento geométrico de razón 1.5 ($k = \log_{1.5}(\frac{n}{C_0}) = \theta(\log n)$)

c) Para la política A tenemos una suma aritmética siendo que el número de *resizes* es lineal en n y el costo de cada uno también crece linealmente, dando la cantidad de copias igual a $\sum_{C=C_0}^{n-1} C \approx \frac{n^2}{2}$ siendo $\theta(n^2)$, para la política B y C son una serie geométrica con $r = 2$ y $r = 1.5$, las dos de $\theta(n)$, se diferencian de que tienen un diferente costo de copiado, esto por que la política C crece mas lento, necesitando mas resizes para llegar a n (siendo casi el doble que en B, $2n$) y el de B, su suma está acotada por su término más grande que en este caso es n y por tanto ese es su costo e cantidad de copias creadas.

d) Sabemos que el costo amortizado es el costo total/ n por lo cual tenemos que para la política A el costo total es $\theta(n) + \theta(n^2) = \theta(n^2)$ y amortizado es $\theta(n)$, para B y C el costo total es $\theta(n) + \theta(n) = \theta(n)$ y amortizado es $\theta(1)$, siendo en C una constante mayor que en B

e) Después de un resize $size = c_{old} + 1$, siendo el espacio libre $c_{new} - (c_{old} + 1)$. Para la política A tenemos $c_{new} - (c_{old} + 1)$ nunca va sobrar espacio por lo tanto es cero, en la política B tenemos que $c_{new} = 2c_{old}$ por lo tanto lo libre es $c_{old} = \frac{c_{new}}{2}$ o es un $\frac{1}{2}$ en fracción y en la política C tenemos que $c_{new} = 1.5c_{old}$ por lo tanto lo libre es $0.5c_{old} = \frac{c_{new}}{3}$ o es un $\frac{1}{3}$ en fracción

f)

Política	#Resizes	Total de copias	Costo amortizado por crecimiento	Espacio sin usar tras resize
C+1	$\theta(n)$	$\theta(n^2)$	$\theta(n)$	0 (0%)
2C	$\theta(\log n)$	$\theta(n) \approx n$	$\theta(1)$	$\approx C/2 (\approx 50\%)$
1.5C	$\theta(\log n)$	$\theta(n) \approx 2n$	$\theta(1)$	$\approx C/3 (\approx 33\%)$

En general, entre más cercano a 1 sea el factor de crecimiento, menos memoria se desperdicia, pero se recopila más y la política aditiva minimiza el desperdicio de memoria (siempre 0) pero paga un costo cuadrático en tiempo.

PREGUNTA FINAL:

Cualquier razón constante $\alpha > 1$ (independiente de n) produce costo amortizado $\theta(1)$ para *addHead*. Lo importante no es duplicar, sino que el crecimiento sea multiplicativo y no aditivo como lo hemos podido ver anteriormente. El total de copias sería:

$$\text{Total de copias } (\alpha, n) \approx n/(\alpha - 1)$$

Cuando $\alpha = 1$, $(\alpha - 1) = 0$ el número de copias tiende al comportamiento cuadrático de la política aditiva en el límite; cuando α crece el número de copias disminuye, pero el espacio desperdiciado aumenta, siendo $\alpha = 2$ un ejemplo, mas no una condición.